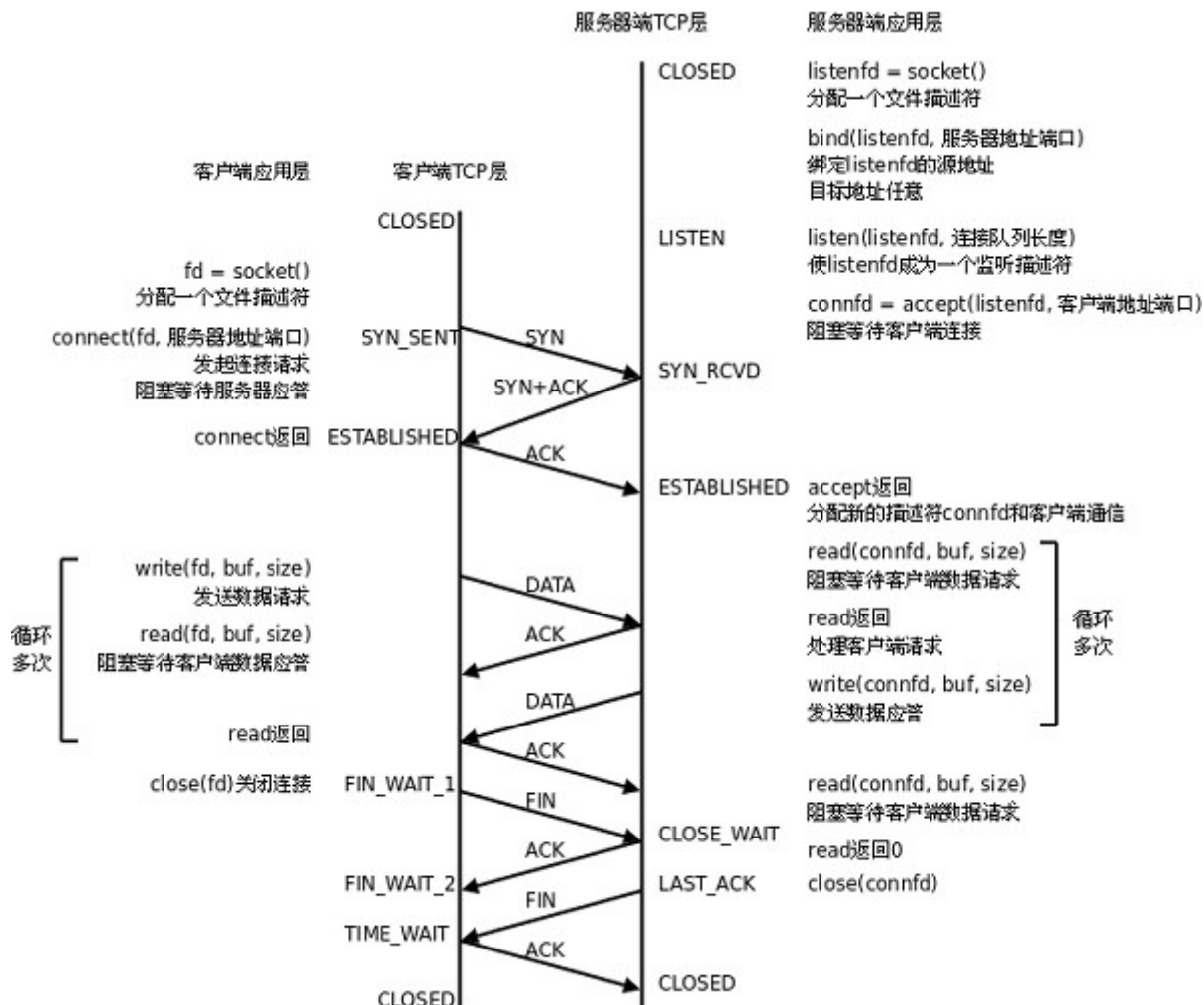


# 网络编程 C/S 模型

## 1、C/S模型-TCP

下图是基于TCP协议的客户端/服务器程序的一般流程:



服务器调用`socket()`、`bind()`、`listen()`完成初始化后, 调用`accept()`阻塞等待, 处于监听端口的状态, 客户端调用`socket()`初始化后, 调用`connect()`发出SYN段并阻塞等待服务器应答, 服务器应答一个SYN-ACK段, 客户端收到后从`connect()`返回, 同时应答一个ACK段, 服务器收到后从`accept()`返回。

数据传输的过程:

建立连接后, TCP协议提供全双工的通信服务, 但是一般的客户端/服务器程序的流程是由客户端主动发起请求, 服务器被动处理请求, 一问一答的方式。因此, 服务器从`accept()`返回后立刻调用`read()`, 读socket就像读管道一样, 如果没有数据到达就阻塞等待, 这时客户端调用`write()`发送请求给服务器, 服务器收到后从`read()`返回, 对客户端的请求进行处理, 在此期间客户端调用`read()`阻塞等待服务器的应答, 服务器调用`write()`将处理结果发回给客户端, 再次调用`read()`阻塞等待下一条请求, 客户端收到后从`read()`返回, 发送下一条请求, 如此循环下去。

如果客户端没有更多的请求了, 就调用 `close()` 关闭连接, 就像管道关闭的管道一样, 服务器的 `read` 返回

如果客户端没有更多的请求了，就调用close()关闭连接，就像与端关闭的管道一样，服务器的read()返回0，这样服务器就知道客户端关闭了连接，也调用close()关闭连接。注意，任何一方调用close()后，连接的两个传输方向都关闭，不能再发送数据了。如果一方调用shutdown()则连接处于半关闭状态，仍可接收对方发来的数据。

在学习socket API时要注意应用程序和TCP协议层是如何交互的：\*应用程序调用某个socket函数时TCP协议层完成什么动作，比如调用connect()会发出SYN段\*应用程序如何知道TCP协议层的状态变化，比如从某个阻塞的socket函数返回就表明TCP协议收到了某些段，再比如read()返回0就表明收到了FIN段

## 1.1.server.c

server.c的作用是从客户端读字符，然后将每个字符转换为大写并回送给客户端。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/socket.h>
6 #include <netinet/in.h>
7 #include <arpa/inet.h>
8
9
10 #define MAXLINE 80
11 #define SERV_PORT 8000
12
13
14 int main(void)
15 {
16     struct sockaddr_in servaddr, cliaddr;
17     socklen_t cliaddr_len;
18
19
20     int listenfd, connfd;
21     char buf[MAXLINE];
22     char str[INET_ADDRSTRLEN];
23     int i, n;
24
25
26     listenfd = socket(AF_INET, SOCK_STREAM, 0);
27
28
29     bzero(&servaddr, sizeof(servaddr));
```

```

30 servaddr.sin_family = AF_INET;
31 servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
32 servaddr.sin_port = htons(SERV_PORT);
33
34
35 bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
36 listen(listenfd, 20);
37 printf("Accepting connections ...\n");
38
39
40 while (1) {
41     cliaddr_len = sizeof(cliaddr);
42     connfd = accept(listenfd, (struct sockaddr *)&cliaddr, &cliaddr_len);
43     n = read(connfd, buf, MAXLINE);
44     printf("received from %s at PORT %d\n", inet_ntop(AF_INET, &cliaddr.sin_addr,
45
46
47     for (i = 0; i < n; i++)
48         buf[i] = toupper(buf[i]);
49     write(connfd, buf, n);
50     close(connfd);
51 }
52 return 0;
53 }

```

## 1.2 client.c

client.c的作用是从命令行参数中获得一个字符串发给服务器，然后接收服务器返回的字符串并打印。

```

1 #include <unistd.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <sys/socket.h>
6 #include <netinet/in.h>
7
8
9 #define MAXLINE 80

```

```

10 #define SERV_PORT 8000
11
12
13 int main(int argc, char *argv[])
14 {
15     struct sockaddr_in servaddr;
16     char buf[MAXLINE];
17     int sockfd, n;
18     char *str;
19     if (argc != 2) {
20         fputs("usage: ./client message\n", stderr);
21         exit(1);
22     }
23
24
25     str = argv[1];
26     sockfd = socket(AF_INET, SOCK_STREAM, 0);
27
28
29     bzero(&servaddr, sizeof(servaddr));
30     servaddr.sin_family = AF_INET;
31     inet_pton(AF_INET, "127.0.0.1", &servaddr.sin_addr);
32     servaddr.sin_port = htons(SERV_PORT);
33
34
35     connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
36
37
38     write(sockfd, str, strlen(str));
39
40
41     n = read(sockfd, buf, MAXLINE);
42     printf("Response from server:\n");
43     write(STDOUT_FILENO, buf, n);
44
45
46     close(sockfd);
47     return 0;
48 }

```

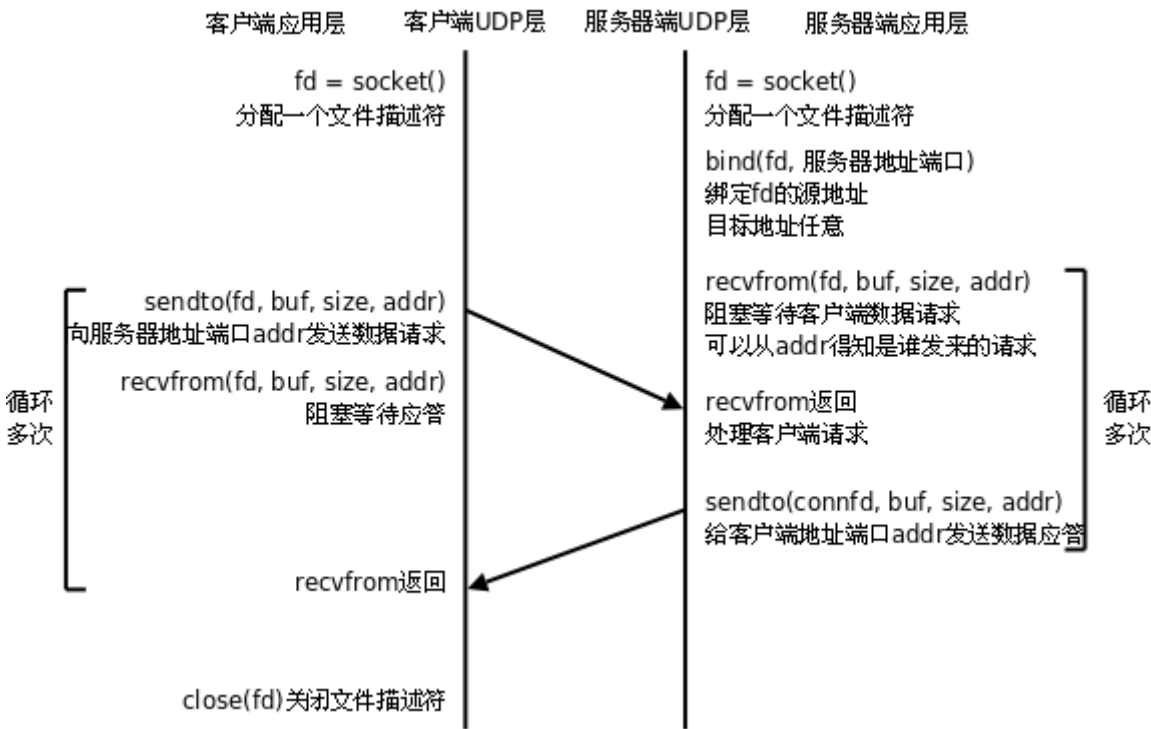
由于客户端不需要固定的端口号，因此不必调用bind()，客户端的端口号由内核自动分配。注意，客户端

不是不允许调用bind(), 只是没有必要调用bind()固定一个端口号, 服务器也不是必须调用bind(), 但如果服务器不调用bind(), 内核会自动给服务器分配监听端口, 每次启动服务器时端口号都不一样, 客户端要连接服务器就会遇到麻烦。

客户端和服务端启动后可以查看链接情况:

```
netstat -apn|grep 8000
```

## 2、C/S模型-UDP



由于UDP不需要维护连接, 程序逻辑简单了很多, 但是UDP协议是不可靠的, 实际上有很多保证通讯可靠性的机制需要在应用层实现。

编译运行server, 在两个终端里各开一个client与server交互, 看看server是否具有并发服务的能力。用Ctrl+C关闭server, 然后再运行server, 看此时client还能否和server联系上。和前面TCP程序的运行结果相比较, 体会无连接的含义。

### 2.1.server.c

```

1 #include <string.h>
2 #include <netinet/in.h>
3 #include <stdio.h>
4 #include <unistd.h>
5 #include <strings.h>
6 #include <arpa/inet.h>
7 #include <ctype.h>
8 #define MAXLINE 80
9 #define SERV_PORT 8000
10
11
12 int main(void)
13 {
14     struct sockaddr_in servaddr, cliaddr;
15     socklen_t cliaddr_len;
16     int sockfd;
17     char buf[MAXLINE];
18     char str[INET_ADDRSTRLEN];
19     int i, n;
20
21
22     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
23
24
25     bzero(&servaddr, sizeof(servaddr));
26     servaddr.sin_family = AF_INET;
27     servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
28     servaddr.sin_port = htons(SERV_PORT);
29
30
31     bind(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
32     printf("Accepting connections ...\n");
33
34
35     while (1) {
36         cliaddr_len = sizeof(cliaddr);
37         n = recvfrom(sockfd, buf, MAXLINE, 0, (struct sockaddr *)&cliaddr, &cliaddr_len);
38         if (n == -1)
39             perror("recvfrom error");
40         printf("received from %s at PORT %d\n", inet_ntop(AF_INET, &cliaddr.sin_addr,
41
42
43             for (i = 0; i < n; i++)

```

```

44         buf[i] = toupper(buf[i]);
45
46
47         n = sendto(sockfd, buf, n, 0, (struct sockaddr *)&cliaddr, sizeof(cliaddr));
48
49
50         if (n == -1)
51             perror("sendto error");
52     }
53
54
55     close(sockfd);
56     return 0;
57 }

```

## 2.2.client.c

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <unistd.h>
4  #include <netinet/in.h>
5  #include <arpa/inet.h>
6  #include <strings.h>
7  #include <ctype.h>
8
9
10 #define MAXLINE 80
11 #define SERV_PORT 8000
12
13
14 int main(int argc, char *argv[])
15 {
16     struct sockaddr_in servaddr;
17     int sockfd, n;
18     char buf[MAXLINE];
19
20
21     sockfd = socket(AF_INET, SOCK_DGRAM, 0);

```

```
22
23
24     bzero(&servaddr, sizeof(servaddr));
25     servaddr.sin_family = AF_INET;
26     inet_pton(AF_INET, "127.0.0.1", &servaddr.sin_addr);
27     servaddr.sin_port = htons(SERV_PORT);
28
29
30     while (fgets(buf, MAXLINE, stdin) != NULL) {
31         n = sendto(sockfd, buf, strlen(buf), 0, (struct sockaddr *)&servaddr, sizeof(
32         if (n == -1)
33             perror("sendto error");
34
35
36         n = recvfrom(sockfd, buf, MAXLINE, 0, NULL, 0);
37
38
39         if (n == -1)
40             perror("recvfrom error");
41
42
43         write(STDOUT_FILENO, buf, n);
44     }
45     close(sockfd);
46
47
48     return 0;
49 }
```